



Centro Regional Universitario Córdoba IUA
Ing. en Informática

SISTEMAS EN TIEMPO REAL

Mag. Ing. José Ducloux
jducloux@iua.edu.ar

2026

Manejo del tiempo con POSIX en C

- Un **temporizador** (timer) permite que un proceso pueda planificar una notificación para sí mismo luego de transcurrir algún tiempo específico.
- El modo **sleep** permite que un proceso o un hilo pueda suspender su ejecución por un periodo de tiempo.
- En Linux se tienen diferentes alternativas:
 - API clásica de UNIX (setitimer y alarm). Son parte de POSIX.
 - API con funciones de sleep (POSIX).
 - Temporizadores de Linux que se leen desde un descriptor de archivo (timerfd). No son POSIX.
 - Relojes y temporizadores POSIX (clocks y timers).

Llamada al sistema alarm()

- Configura la señal SIGALRM para notificar al proceso que realiza la llamada al transcurrir una determinada cantidad de segundos por única vez.
- Para uso periódico, se debe llamar nuevamente una vez recibida la señal.

```
#include <unistd.h>
```

```
unsigned int alarm(unsigned int seconds);
```

- seconds: segundos que transcurren desde que se llama a la función y se recibe la señal SIGALRM.
- Retorno: devuelve la cantidad de segundos que quedan por transcurrir de una alarma previamente programada, o 0 si no había ninguna alarma previamente programada.

sleep() y usleep()

- **sleep()**

Suspende un proceso o hilo durante una determinada cantidad de segundos.

- `#include <unistd.h>`
- `unsigned int sleep(unsigned int seconds);`
seconds: especifica los segundos a permanecer en modo suspendido.
Retorno: 0 si se cumplió el tiempo de suspensión, o los segundos que quedan por transcurrir en caso de salir del modo de suspensión por otro motivo.

- **usleep()**

Similar a sleep() pero en lugar de usar segundos utiliza como unidad de tiempo microsegundos.

- `int usleep(useconds_t usec);`
usec: especifica los microsegundos a permanecer en modo suspendido.

Relojes (clocks) de POSIX

- POSIX proporciona una API para acceder a relojes que miden el tiempo con precisión de nanosegundos.
- Se debe enlazar con la biblioteca de tiempo real `rt`.
- Las principales funciones de la API son:
 - `clock_gettime()`: retorna el valor actual de un reloj.
 - `clock_getres()`: retorna la resolución de un reloj.
 - `clock_settime()`: actualiza un reloj.

Relojes de POSIX

- Tipos de relojes
 - `CLOCK_REALTIME`: reloj configurable de tiempo real del sistema.
 - `CLOCK_MONOTONIC`: reloj no configurable.
 - `CLOCK_PROCESS_CPUTIME_ID`: reloj que mide el tiempo consumido por un proceso.
 - `CLOCK_THREAD_CPUTIME_ID`: reloj que mide el tiempo consumido por un hilo dentro de un proceso.

Temporizadores de intervalos de POSIX

- Características
 - Resolución en nanosegundos.
 - Los temporizadores POSIX no los hereda un hijo creado por `fork()`.
 - Se desarmen y eliminan durante un `exec()` o al finalizar el proceso.
- Se debe enlazar con la biblioteca `rt`.
- Funciones de la API:
 - `timer_create()`: crea un timer y define el método por el cual se notifica al proceso cuando expira.
 - `timer_settime()`: inicia y/o detiene un timer.
 - `timer_gettime()`: retorna el intervalo y el tiempo restante para la próxima expiración del timer.
 - `timer_delete()`: elimina un timer.

Temporizadores de intervalos de POSIX

- Creación del timer
 - `int timer_create(clockid_t clockid, struct sigevent *evp, timer_t *timerid);`
- Argumentos
 - `clockid`: identificador de reloj a ser usado por el timer.
 - `*evp`: estructura que determina cómo el proceso será notificado cuando expire el timer.
 - `sigev_notify`: tipo de notificación.
 - `sigev_signo`: número de señal.
 - `sigev_value.sival_ptr`: identificador del timer.
 - `*timerid`: identificador del timer.
- Retorna 0 si se crea el timer, -1 si hubo un error.

Clock ID
CLOCK_REALTIME
CLOCK_MONOTONIC
CLOCK_PROCESS_CPUTIME_ID
CLOCK_THREAD_CPUTIME_ID

<i>sigev_notify</i> value
SIGEV_NONE
SIGEV_SIGNAL
SIGEV_THREAD

Temporizadores de intervalos de POSIX

- Armado y desarmado del timer
 - `int timer_settime(timer_t timerid, int flags, const struct itimerspec *value, struct itimerspec *old_value);`
- Argumentos
 - `timerid`: identificador del timer.
 - `flags`: si es 0, el argumento `value.it_value` se interpreta en relación con el valor del reloj en el momento de la llamada a `timer_settime()`. Si `flags` se pasa como `TIMER_ABSTIME` entonces `value.it_value` se interpreta como tiempo absoluto medido desde el punto cero del reloj.
 - `value`: especifica la nueva configuración de tiempos del timer.
 - `old_value`: contiene la configuración previa del timer.
 - `value` y `old_value` apuntan a estructuras `itimerspec`.
 - El campo `it_value` especifica la expiración inicial del timer por única vez. Si el subcampo `it_interval` es distinto de cero, entonces el timer es periódico luego de darse la expiración inicial.
 - La estructura `itimerspec` contiene estructuras `timespec`.
- Para armar y desarmar el timer se llama a `timer_settime()` con los subcampos de `value.it_value` distintos de cero (al menos uno ellos).
- Si los valores de tiempos no son múltiplos de la resolución del reloj correspondiente, se redondean.

```
struct itimerspec {
    struct timespec it_interval; /* Interval for periodic timer */
    struct timespec it_value;    /* First expiration */
};
```

```
struct timespec {
    time_t tv_sec; /* Seconds */
    long tv_nsec; /* Nanoseconds */
};
```

Temporizadores de intervalos de POSIX

- Notificación por medio de una señal
 - Cuando el timer expira, se puede aceptar una señal a través del controlador de señal asignado.
 - Se debe asignar el controlador de señal que será usado por el timer cuando expire.
 - La forma más simple es usar la función *signal()* que recibe la señal y el controlador de señal.
 - Otra forma más detallada:
 - Se usa la estructura *sigaction* para configurar la señal
 - *sa_flags*: definen el comportamiento de la señal.
 - *sa_sigaction*: controlador de señal.
 - Con la función *sigemptyset()* se inicializa la señal excluyendo a las demás.
 - Con la función *sigaction()* se configura la acción que tomará un proceso al recibir una señal.
 - Se le pasan como argumentos la estructura *sigaction* y la señal.



Centro Regional Universitario Córdoba IUA
Ing. en Informática

¿Preguntas?